



6.2.5 Conditioning versus stability ¶

6.2.5 Stability and Conditioning

fit width



It is important to keep conditioning versus stability straight:

- *Conditioning* is a property of the problem you are trying to solve. A problem is well-conditioned if a small change in the input is guaranteed to only result in a small change in the output. A problem is ill-conditioned if a small change in the input can result in a large change in the output.
- *Stability* is a property of an implementation. If the implementation, when executed with an input always yields an output that can be attributed to slightly changed input, then the implementation is backward stable.

In other words, in the presence of roundoff error, computing a wrong answer may be due to the problem (if it is ill-conditioned), the implementation (if it is numerically unstable), or a programming bug (if the implementation is sloppy). Obviously, it can be due to some combination of these.

Now,

- If you compute the solution to a well-conditioned problem with a numerically stable implementation, then you will get an answer that is close to the actual answer.
- If you compute the solution to a well-conditioned problem with a numerically unstable implementation, then you may or may not get an answer that is close to the actual answer.
- If you compute the solution to an ill-conditioned problem with a numerically stable implementation, then you may or may not get an answer that is close to the actual answer.

Yet another way to look at this: A numerically stable implementation will yield an answer that is as accurate as the conditioning of the problem warrants.